

Lab: Differential Expression via RNA-Seq Analysis

Genomic Technologies Workshop 2026 (PLPTH885)

Sanzhen Liu

6/10/2026

Outline

- Differential expression test using DESeq2
- Result visualization

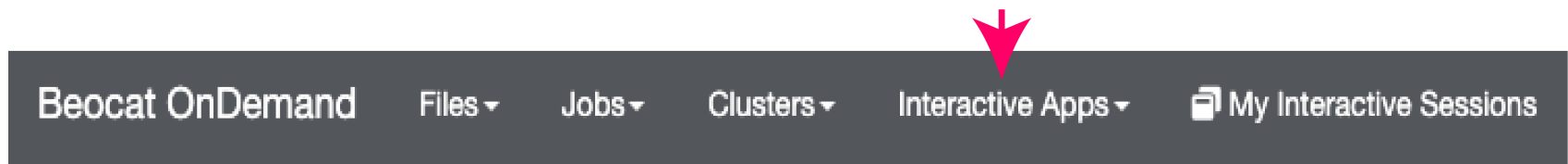
Course webpage

RNA-seq DE analysis

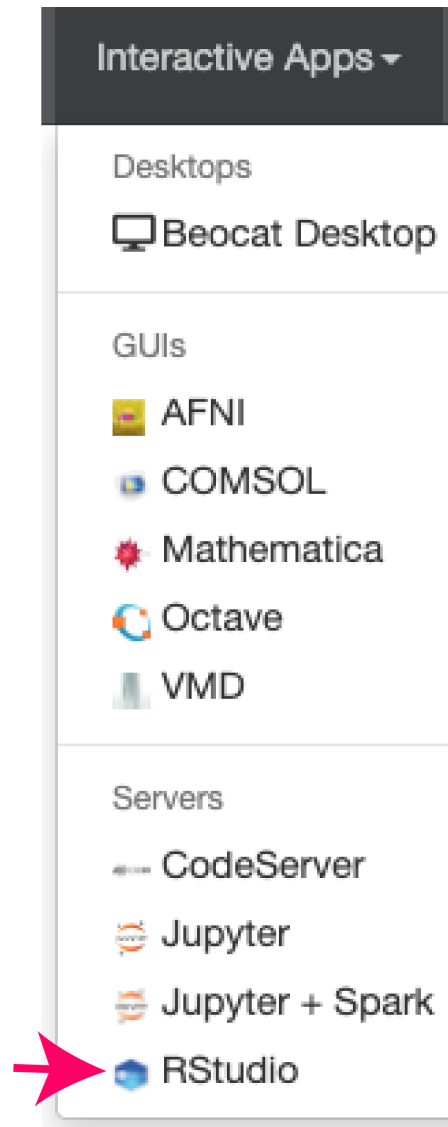
OnDemand at Beocat

ondemand

login with eID



Select RStudio



Request resources

RStudio

This app will launch [RStudio Server](#) an IDE for [R](#).

R version

4.3.2 (foss-2023a)



This defines the version of R you want to load.

Number of hours

24

Number of cores

1

Amount of memory

16

The amount of memory (in GB) needed for the whole job

Job Type

normal



Connect to RStudio

RStudio (5655555)1 node | 1 core | Running

Host: >_wizard24.beocat.ksu.edu

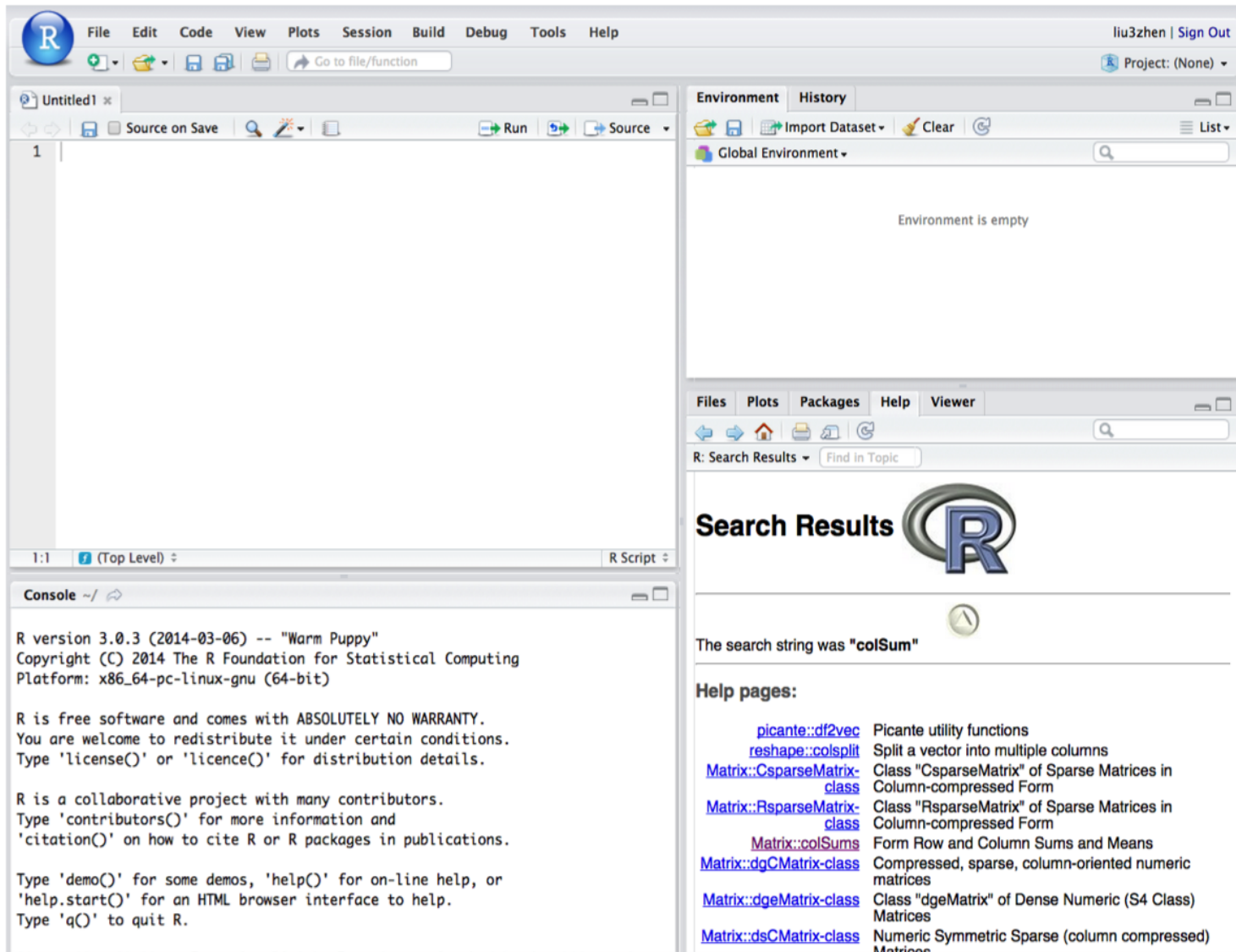
Created at: 2022-06-05 13:25:22 CDT

Time Remaining: 2 hours and 59 minutes

Session ID: 8039a538-a703-42c0-9d6f-47a3a0c0a1c2

[® Connect to RStudio Server](#)[Delete](#)

Rstudio interface



Package installation

DESeq2 installation

```
if (!require("BiocManager", quietly=T))  
  install.packages("BiocManager")  
  
if (!require("DESeq2", quietly=T))  
  BiocManager::install("DESeq2") # DESeq2
```

ClusterProfiler

```
if (!require("ggplot2", quietly=T)) {  
  biocmanger_version <- BiocManager::version()  
  if (biocmanger_version == "3.18") {  
    install.packages("https://cran.r-project.org/src/contrib/Archive,  
      repos = NULL, type = "source")  
  } else {  
    install.packages("ggplot2")  
  }  
}  
  
if (!require("clusterProfiler", quietly=T))  
  BiocManager::install("clusterProfiler")
```


preload modules

```
githubpage <- "https://raw.githubusercontent.com/liu3zhenlab/teaching  
data_url <- paste0(githubpage, "/master/RNA-Seq-Workshop/2026/lab_DE'  
preload_r <- paste0(data_url, "/utils/load.R")  
  
source(preload_r)
```

- panel.cor2
- rnaseq.pca
- normalization

codes

Read expression data (Read counts per gene)

```
rc_file <- paste0(data_url, "/data/rc.txt")
grc <- read.delim(rc_file)
nrow(grc)  # the number of rows/lines
```

[1] 19999

first entry:

Gene	trt1	trt2	trt3	ctl1	ctl2	ctl3
Zm00001eb000010	283	191	166	308	268	114

last entry:

Gene	trt1	trt2	trt3	ctl1	ctl2	ctl3
Zm00001eb222700	0	0	0	0	0	0

RPKM normalization

```
datacols <- c("trt1", "trt2", "trt3",  
             "ctl1", "ctl2", "ctl3")  
lib.sizes <- colSums(grc[, datacols]) # library sizes  
grcn <- normalization(counts = grc,  
                      normcolname = datacols,  
                      libsize = lib.sizes,  
                      methods = "RPM")
```

Gene	trt1	trt2	trt3	ctl1	ctl2	ctl3	trt1.RPM	trt2.RPM	trt3.RPM	ctl1.RPM	ctl2.RPM	ctl3.RPM
Zm00001eb000010	283	191	166	308	268	114	34.888	20.814	20.813	26.143	30.537	14.137
Zm00001eb000020	264	190	199	302	174	208	32.546	20.705	24.950	25.633	19.826	25.795

data organization for DESeq2

- count information

```
geneid <- grc$Gene  
in.data <- as.matrix(grc[, 2:7])
```

trt1	trt2	trt3	ctl1	ctl2	ctl3
283	191	166	308	268	114
264	190	199	302	174	208
0	0	0	0	0	0

sample names and grouping information (treatment)

```
sample.ids <- colnames(in.data)
treatment <- c("trt", "trt", "trt", "ctl", "ctl", "ctl")
sample.info <- data.frame(row.names=sample.ids, trt=treatment)
```

	trt
trt1	trt
trt2	trt
trt3	trt
ctl1	ctl
ctl2	ctl
ctl3	ctl

Differential expression test

```
dds <- DESeqDataSetFromMatrix(countData=in.data,  
                               colData=sample.info,  
                               formula(~trt))  
dds <- DESeq(dds)
```

DE output

```
res <- results(object = dds)
res <- data.frame(res)
res$Gene <- geneid
res <- res[,c("Gene", "baseMean", "log2FoldChange", "pvalue", "padj")]
nrow(res)
```

```
[1] 19999
```

DE + normalized data

```
### Merge the normalized result with the DE result  
out <- merge(grcn, res, by = "Gene")  
out <- data.frame(out)
```

Gene	trt1	trt2	trt3	ctl1	ctl2	ctl3
Zm00001eb000010	283	191	166	308	268	114
Zm00001eb000020	264	190	199	302	174	208

trt1.RPM	trt2.RPM	trt3.RPM	ctl1.RPM	ctl2.RPM	ctl3.RPM
34.888	20.814	20.813	26.143	30.537	14.137
32.546	20.705	24.950	25.633	19.826	25.795

baseMean	log2FoldChange	pvalue	padj
217.4257	-0.0277417	0.9342927	0.9777347
219.4751	0.0048971	0.9853634	0.9954542

significant gene sets at different FDRs

```
sum(!is.na(out$padj) & out$padj < 0.05)
```

```
[1] 1571
```

problem

Please revise the code to calculate the number of significant genes with the FDR smaller than 10% and 15%?

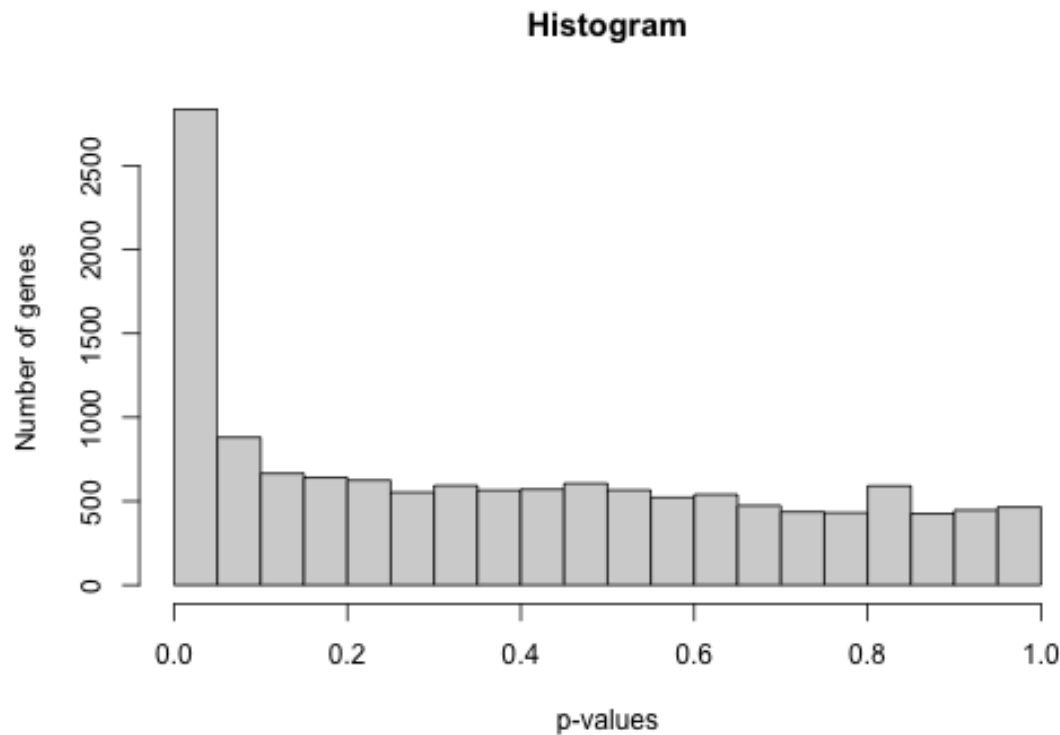
significantly DEG

```
sig <- out[!is.na(out$padj) & out$padj < 0.05, ]  
kable(sig[1:2, c(1, 8, 11, 15:ncol(sig))]) %>%  
  kable_styling(font_size = 16,  
                position = "left",  
                full_width = F)
```

	Gene	trt1.RPM	ctl1.RPM	log2FoldChange	pvalue	padj
29	Zm00001eb000370	5.917	49.399	-2.7481125	0.00e+00	0.0000000
34	Zm00001eb000420	40.313	77.409	-0.8738775	1.78e-05	0.0003425

p-value histogram

```
pvals <- out$pvalue  
hist(pvals, main="Histogram",  
      xlab="p-values",ylab="Number of genes")
```

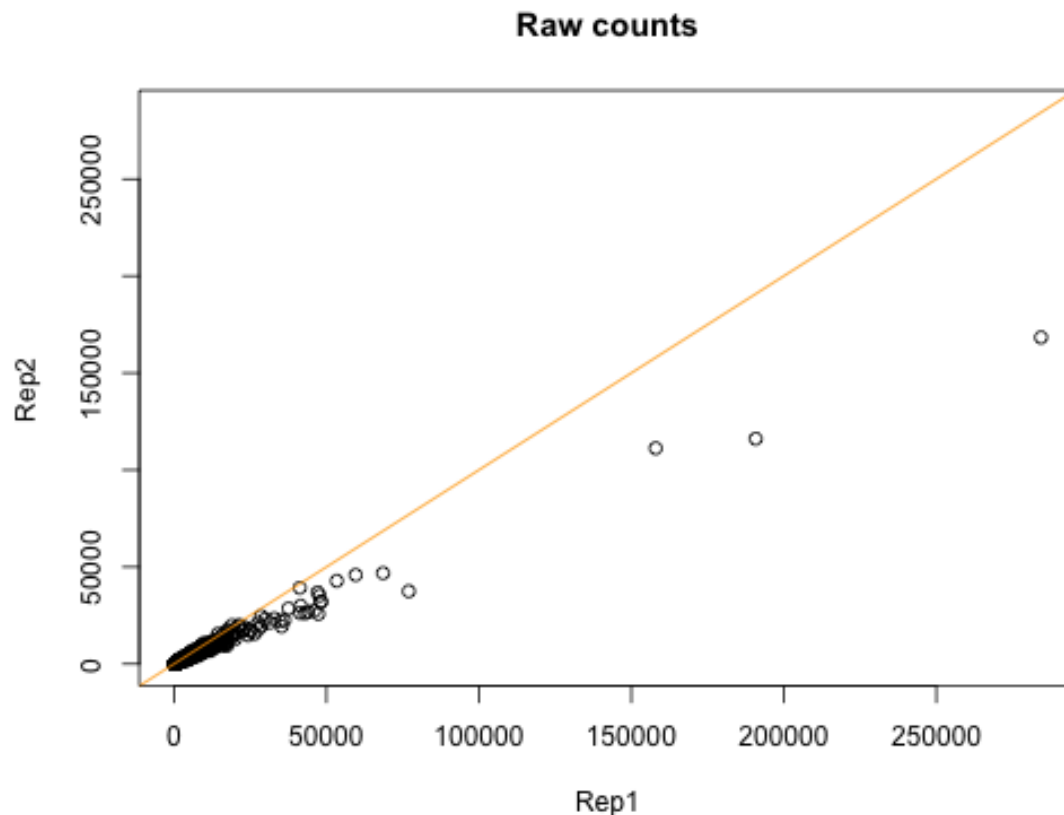


problem

Please modify the plot code to change the figure title to "DE"

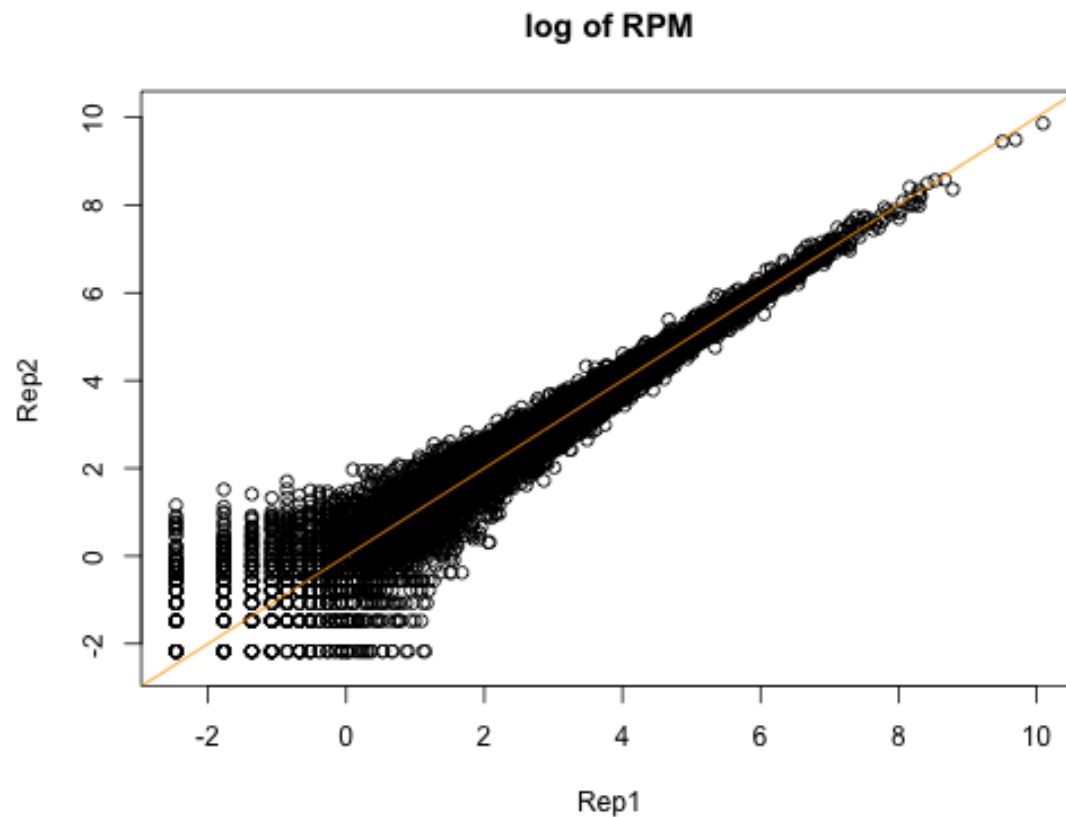
scatter plot - raw counts

```
x.raw <- out$ctl1; y.raw <- out$ctl2  
drange <- range(x.raw, y.raw, finite=T)  
# plot a scatter plot between two samples  
plot(x.raw, y.raw, main="Raw counts", xlab="Rep1",  
      ylab="Rep2", xlim=drange, ylim=drange)  
abline(a = 0, b = 1, col = "orange")
```



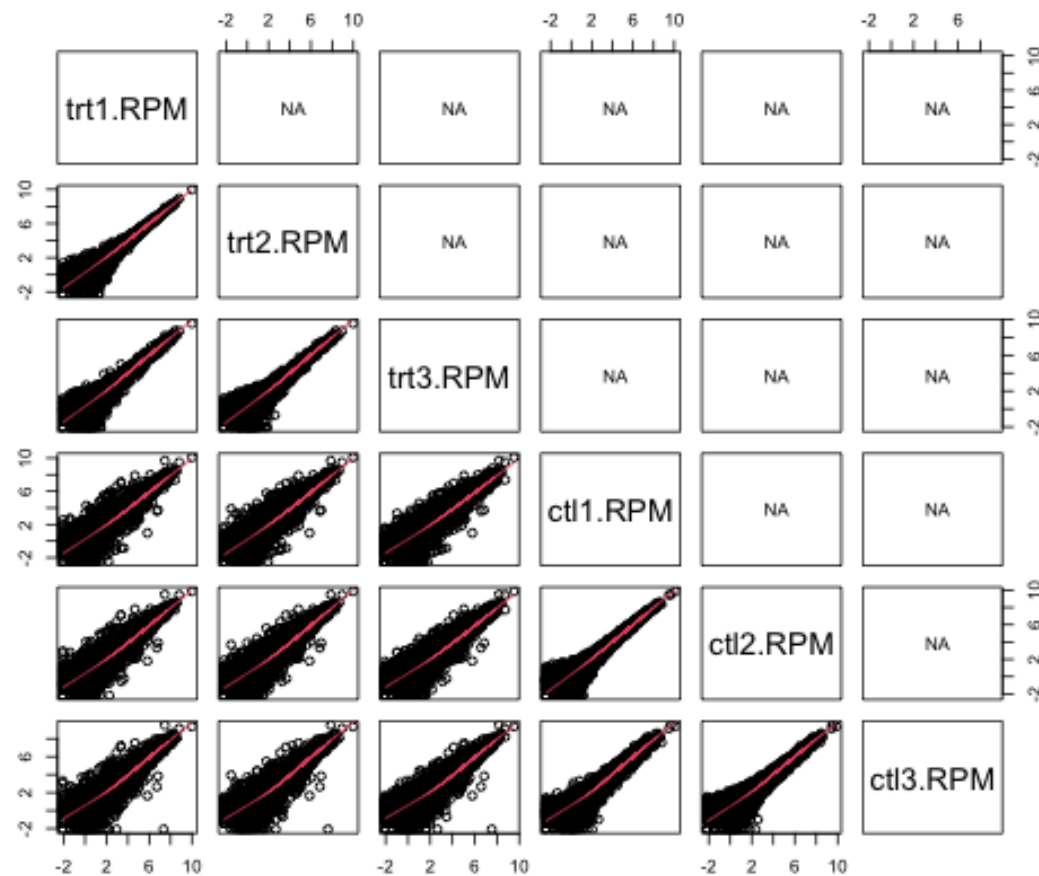
scatter plot - RPM

```
x <- log(out$ctl1.RPM); y <- log(out$ctl2.RPM)
drange <- range(x, y, finite=T)
plot(x, y, main="log of RPM", xlab="Rep1",
      ylab="Rep2", xlim=drange, ylim=drange)
abline(a = 0, b = 1, col="orange")
```



pair-wise scatter plots

```
logrpm <- log(out[, 8:13])  
pairs(logrpm, lower.panel=panel.smooth, upper.panel=panel.cor2)
```

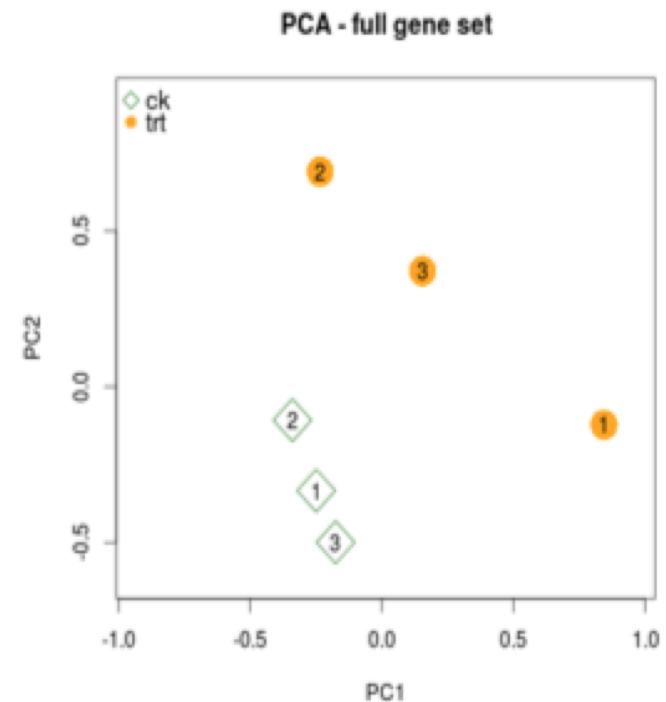


Principal Component Analysis (PCA)

PCA is a mathematical algorithm that reduces the dimensionality of the data while retaining most of the variation in the data set.

Control				Treatment		
Gene	Rep1	Rep2	Rep3	Rep1	Rep2	Rep3
1	2679	2360	2573	2563	3398	3012
2	177	161	171	154	137	152
3	381	371	397	541	723	635
...						
20000	990	1073	1236	850	672	859

Normalized and standardized data



function / module

You can write your own function:

```
fun_name <- function (...) { ... }
```

```
gpa_improve <- function(gpa, rate) {  
  ### gpa: a numeric vector for GPAs  
  ### rate: percentage for the improvement  
  new.gpa <- gpa * (1 + rate)  
  new.gpa[new.gpa > 4] <- 4  
  return(new.gpa)  
}
```

```
### running the function  
our.gpa <- c(3.8, 3.3, 2.8, 3.1)  
gpa_improve(our.gpa, 0.1)
```

```
[1] 4.00 3.63 3.08 3.41
```

```
gpa_improve(our.gpa, 0.2)
```

```
[1] 4.00 3.96 3.36 3.72
```

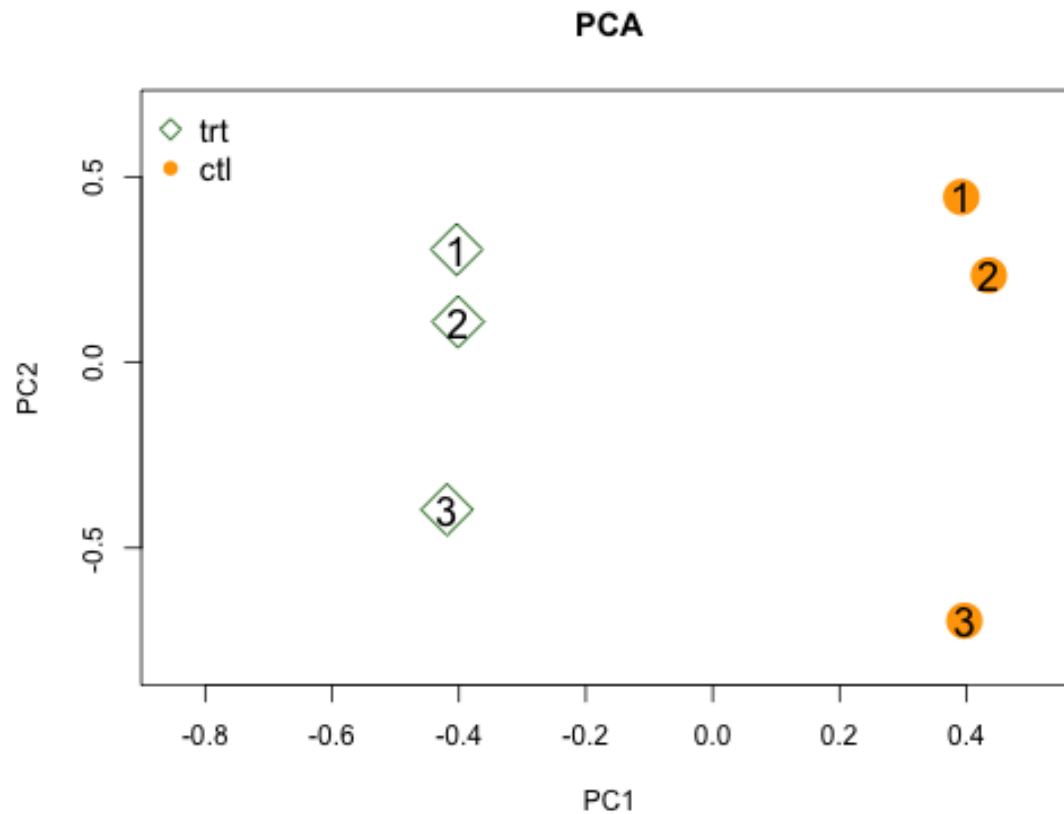

PCA function

principal component analysis and plotting

```
rnaseq.pca <- function(norm.data,  
  norm.feature="RPM",  
  group.feature,  
  title="",  
  shape.code=NULL,  
  mean.cutoff=0.1,  
  colors=NULL,  
  scaling=T, ...) {  
  ...  
}
```

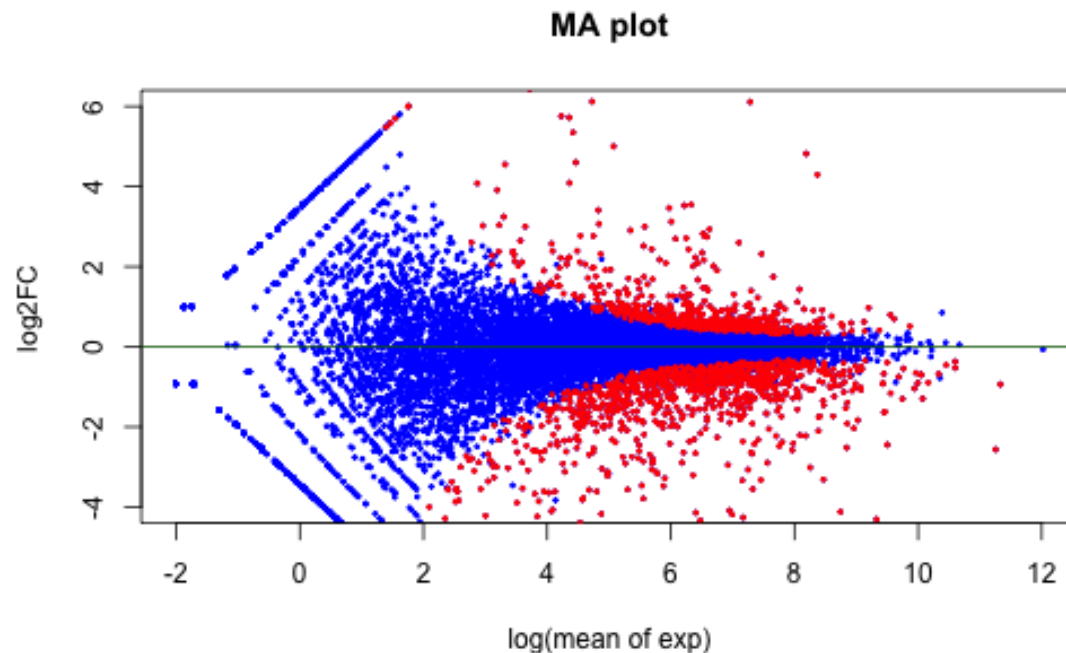
PCA plotting

```
rnaseq.pca(out, norm.feature="RPM",  
  group.feature=c("trt", "ctl"), shape.code=c(5, 16),  
  mean.cutoff=0.1, cex=3, scaling = T,  
  title="PCA", colors=c("dark green","orange"))
```



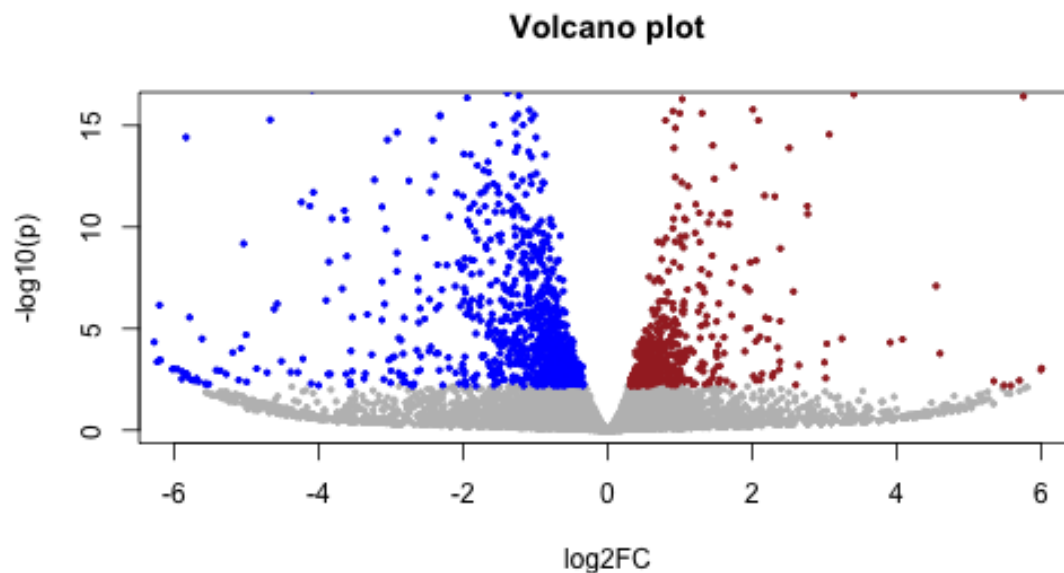
MA plot

```
aval <- log(out$baseMean)
mval <- out$log2FoldChange
sig.set <- which(out$padj < 0.05)
plot(aval, mval, main="MA plot", xlab="log(mean of exp)",
     ylab="log2FC", ylim=c(-4,6),pch=19,cex=0.3,col="blue")
# add points and a horizontal line
points(aval[sig.set], mval[sig.set], pch=19,cex=0.3,col="red")
abline(h = 0, cex = 1, col = "dark green")
```



Volcano plot

```
log2val <- out$log2FoldChange # log2 fold change
mlogP <- -log10(out$pvalue) # transformed p-values
### plot
plot(log2val, mlogP, main="Volcano plot", xlab="log2FC",
     ylab="-log10(p)", pch=19, cex=0.4, col="grey", xlim=c(-6,6), ylim=c(0,16))
### highlight
up <- which(out$padj < 0.05 & log2val > 0) # up
dn <- which(out$padj < 0.05 & log2val < 0) # down
points(log2val[up], mlogP[up], pch=19, cex=0.4, col="brown")
points(log2val[dn], mlogP[dn], pch=19, cex=0.4, col="blue")
```



Gene Ontology enrichment - package installation

clusterProfiler

```
library(GO.db)  
library(ggplot2)  
library(clusterProfiler)
```

GO database

```
# GO DB (2 columns)
go <- read.delim("https://raw.githubusercontent.com/liu3zhenlab/teach
colnames(go) <- c("Gene", "GO")
go <- go[, c("GO", "Gene")] # switch gene and GO
is_go_valid <- go$GO %in% keys(GOTERM)
go <- go[is_go_valid, ]
```

GO	Gene
GO:0003690	Zm00001eb000010
GO:0006353	Zm00001eb000010
GO:0006355	Zm00001eb000010
GO:0009507	Zm00001eb000010
GO:0032502	Zm00001eb000010
GO:0000712	Zm00001eb000020
GO:0005634	Zm00001eb000020
GO:0005737	Zm00001eb000020

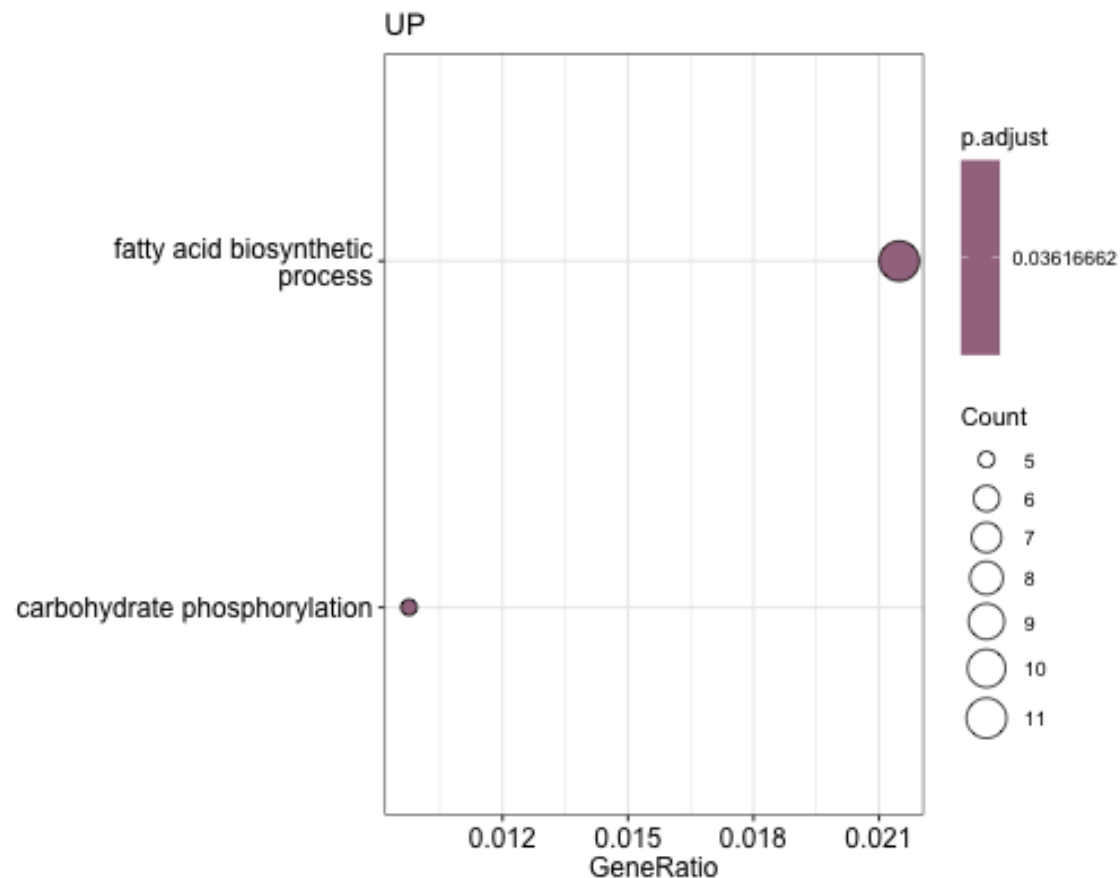
DEG (up- and down-regulated)

```
up_gene <- out$Gene[up]  
dn_gene <- out$Gene[dn]
```

In total, 572 and 999 up- and down-regulated genes were identified, respectively.

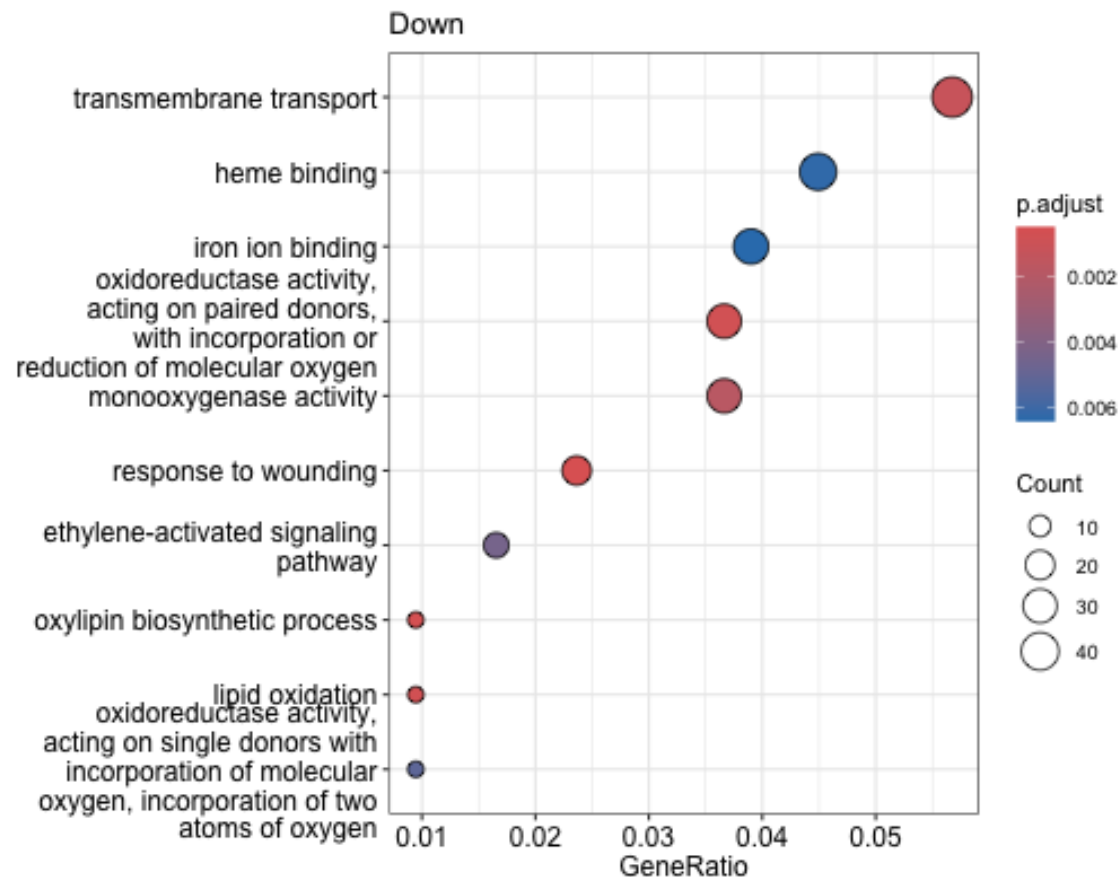
GO enrichment test

```
up_enrich <- enricher(up_gene, universe=out$Gene, TERM2GENE=go)
up_enrich@result$Description <- Term(GOTERM[up_enrich@result$ID])
up_dp <- dotplot(up_enrich, showCategory=10, title="UP", x="GeneRatio")
up_dp + aes(y = Description)
```



Significant down-regulated

```
dn_enrich <- enricher(gene=dn_gene, universe=out$Gene, TERM2GENE=go)
dn_enrich@result$Description <- Term(GOTERM[dn_enrich@result$ID])
dn_dp<-dotplot(dn_enrich,showCategory=10,title="Down",x="GeneRatio")
dn_dp + aes(y = Description)
```



Summary of the analyzing procedure

1. Read counts per gene
2. DE analysis based on the experimental design
3. Examine results (p-value distribution, number of significant genes)
4. Gene Ontology enrichment test

DESeq2 tutorial

Contact information

Sanzhen Liu
Plant Pathology
4729 Throckmorton Plant Sciences Center
Manhattan, KS 66506-5502
phone: 785-532-1379

liu3zhen@ksu.edu

Bioinformatics Applications

PLPTH813, Spring 2027